

Programación orientada a objetos y manejo de eventos en JavaScript

Estudiante:

Carlos Ademar Suarez

Universidad Privada Domingo Savio

Carrera de Ingeniería de Sistemas

Materia:

Diseño Web 2

Docente:

Pablo Alvaro Moscoso

Santa Cruz de la Sierra-Bolivia

2026

Contenido

Introducción....4

Propósito y alcance....5

Objetivo general....5

Objetivos específicos....5

Marco teórico....6

Clases y objetos....6

Módulos y separación de responsabilidades....7

Eventos y actualización del DOM....7

Arquitectura cliente servidor....7

Base de datos relacional e integridad....7

API y persistencia de información....8

Organización e implementación....8

Validación de los campos....8

Manejo de eventos y eliminación....9

Registro y presentación....10

Eliminación individual....10

Comprobación del funcionamiento....10

Desarrollo e integración con Neon y Render....12

Persistencia de productos en PostgreSQL....12

Comunicación entre la interfaz y el servidor 12

Preparación y publicación de la aplicación 13

Limitaciones y mejoras 26

Ejes transversales y conclusiones 26

Aprendizaje y uso responsable de inteligencia artificial 26

Relación educativa y social 27

Conclusiones 28

Referencias 29

Introducción

El taller desarrolla un formulario de productos con HTML, CSS y JavaScript. Su finalidad es aplicar programación orientada a objetos y eventos para comprobar los datos antes de registrarlos. La solución organiza la información en objetos, muestra mensajes de error y permite eliminar productos de la lista.

El trabajo documenta la construcción progresiva del formulario, su división en módulos y la incorporación de acciones mediante teclado y mouse. La separación entre modelo, validación y controlador facilita comprender qué hace cada parte y modificarla sin concentrar toda la lógica en un solo archivo.

Propósito y alcance

Desarrollar módulos lógicos orientados a objetos y controladores de eventos de teclado y mouse para la validación interactiva del formulario de productos. El alcance incluye nombre, precio, cantidad, cálculo del total, registro, limpieza y eliminación individual.

La versión final utiliza HTML, CSS y JavaScript en el navegador, un servidor Node.js con Express y una base de datos PostgreSQL alojada en Neon. Los productos se consultan, registran y eliminan mediante una API. Los datos permanecen en la base de datos al recargar la página.

Objetivo general

Desarrollar una aplicación web de registro de productos mediante programación orientada a objetos y manejo de eventos en JavaScript, con validación de datos, persistencia en PostgreSQL mediante Neon y publicación en Render, para gestionar el registro, la consulta y la eliminación de productos desde una interfaz accesible.

Objetivos específicos

- Representar cada producto mediante una clase con atributos y un método de cálculo.
- Separar las reglas de validación de la presentación de los resultados.
- Responder a eventos input, submit, click y keydown.
- Organizar el código en módulos reutilizables con import y export.
- Definir casos de prueba que permitan comprobar registros válidos, errores y eliminación.
- Implementar una API con Node.js y Express para consultar, registrar y eliminar productos mediante consultas parametrizadas.
- Configurar la base de datos PostgreSQL en Neon con restricciones de integridad para nombre, precio y cantidad.

- Incorporar una confirmación de eliminación que permita aceptar o cancelar la operación antes de modificar la base de datos.
- Publicar la aplicación desde GitHub en Render y documentar la configuración y las evidencias del despliegue.

El marco teórico reúne los conceptos que sustentan la estructura del programa y la relación entre la interfaz, el servidor y la base de datos. Estos fundamentos permiten explicar cómo se representan los productos, cómo se procesan las acciones del usuario y cómo se conserva la información.

Marco teórico

Clases y objetos

Una clase define la estructura y los métodos que comparten sus instancias. El constructor inicializa el objeto al utilizar new. En el proyecto, Producto agrupa nombre, precio y cantidad; calcularTotal devuelve el resultado de multiplicar precio por cantidad. Esta organización relaciona los datos con el comportamiento que les corresponde (MDN Web Docs, s. f.-a).

```
export class Producto {
    constructor(nombre, precio, cantidad) {
        this.nombre = nombre;
        this.precio = precio;
        this.cantidad = cantidad;
    }

    calcularTotal() {
        return this.precio * this.cantidad;
    }
}
```

Por ejemplo, new Producto("Cuaderno", 15.50, 2) crea una instancia cuyo total es 31. La clase no lee campos HTML ni muestra mensajes: esas tareas pertenecen al controlador. No se utiliza herencia porque las necesidades del ejercicio se resuelven mediante clases con responsabilidades distintas.

Módulos y separación de responsabilidades

Los módulos permiten exportar una clase e importarla donde se necesita. El punto de entrada carga el controlador, y este importa el modelo y el validador. El HTML utiliza type="module". Los módulos se ejecutan en modo estricto y deben probarse mediante un servidor local para evitar las restricciones de carga desde archivos locales (MDN Web Docs, s. f.-c).

```
import { ControladorProductos }
    from "./ControladorProductos.js";

new ControladorProductos();
```

Eventos y actualización del DOM

Un evento representa una acción detectada por el navegador, como escribir, enviar un formulario o pulsar un botón. addEventListener permite asociar una función a esa acción. El DOM representa los elementos del documento y permite actualizar los mensajes y la lista sin recargar toda la página (MDN Web Docs, s. f.-b).

Arquitectura cliente servidor

La interfaz recoge los datos y envía solicitudes HTTP al servidor. El servidor comprueba los valores, ejecuta las operaciones de datos y devuelve una respuesta. Esta separación concentra las credenciales y el acceso a PostgreSQL fuera del navegador.

Base de datos relacional e integridad

Los productos se organizan en filas de una tabla. La clave primaria identifica cada registro, mientras que las restricciones NOT NULL y CHECK establecen condiciones de integridad. El tipo NUMERIC permite almacenar el precio con dos posiciones decimales. Neon aloja la base de datos utilizada por el proyecto.

API y persistencia de información

La API define las rutas de comunicación entre las partes del sistema. GET consulta los productos, POST crea un registro y DELETE elimina el registro identificado. La persistencia permite recuperar la información después de recargar la interfaz, a diferencia del arreglo temporal de la versión inicial.

Organización e implementación

Archivo	Responsabilidad
index.html	Campos, etiquetas, botones y lista de productos.
estilos.css	Diseño, distribución y estados visuales de error.
js/Producto.js	Atributos del producto y cálculo del total.
js/ValidadorProducto.js	Reglas para nombre, precio y cantidad.
js/ControladorProductos.js	Eventos, lectura del formulario y actualización del DOM.
js/app.js	Creación de la instancia que inicia la aplicación.

Validación de los campos

El validador recibe el nombre del campo y su contenido. Primero elimina espacios de los extremos para detectar entradas vacías. Después convierte los campos numéricos con

Number y comprueba que tengan valores adecuados. Una cadena vacía como respuesta representa ausencia de error; cualquier otro texto es el mensaje que debe mostrarse.

Campo	Regla aplicada
Nombre	Obligatorio; no se acepta una entrada formada solo por espacios.
Precio	Número finito mayor que cero. El controlador consulta stepMismatch con step="0.01" para limitar los decimales.
Cantidad	Entero seguro mayor o igual que cero. La cantidad cero es válida.

validarCampo muestra el mensaje junto al input y actualiza aria-invalid. Antes de registrar, validarFormulario revisa todos los campos y coloca el foco en el primero incorrecto. También se comprueba que el total calculado sea finito para rechazar valores extremos.

Manejo de eventos y eliminación

addEventListener asocia una función con un evento de un elemento. El controlador registra estas funciones una vez al iniciar la aplicación. Las funciones flecha permiten conservar la referencia this del controlador dentro de las respuestas a los eventos (MDN Web Docs, s. f.-b).

Evento	Acción en el proyecto

input	Valida al escribir, pegar o modificar un campo.
submit	Procesa Registrar o Enter desde un campo y evita recargar mediante preventDefault.
click	Activa los botones Limpiar y Eliminar.
keydown	Detecta Escape y limpia el formulario.

Registro y presentación

Después de validar, el controlador obtiene los datos y crea un objeto Producto. Lo incorpora al arreglo productos con push y genera un elemento li. textContent permite presentar el nombre como texto, sin interpretarlo como HTML. Finalmente, limpia los campos y comunica que el registro fue realizado.

Eliminación individual

Cada fila incorpora un botón Eliminar asociado al objeto que representa. El método localiza ese objeto con indexOf, lo retira del arreglo con splice y elimina su fila del DOM. Esta asociación permite distinguir productos aunque tengan el mismo nombre.

```
eliminarProducto(producto, elemento) {
    const posicion = this.productos.indexOf(producto);
    if (posicion === -1) return;

    this.productos.splice(posicion, 1);
    elemento.remove();
    this.mensaje.textContent =
        "Producto eliminado correctamente.";
}
```

Comprobación del funcionamiento

La captura compartida durante el desarrollo muestra un producto denominado lápiz con precio Bs 15.00, cantidad 25 y total Bs 375.00. Esa evidencia corresponde al registro previo a la

incorporación del botón Eliminar. No demuestra por sí sola la ejecución de todos los casos de validación ni de la eliminación posterior.

La siguiente matriz especifica pruebas manuales y resultados esperados. Deben ejecutarse sobre la versión final en el navegador; no se presentan como pruebas ya aprobadas.

Caso	Resultado esperado
Registrar campos vacíos	Mostrar errores y no agregar productos.
Nombre con espacios	Rechazar el nombre.
Precio 0 o negativo	Solicitar un precio mayor que cero.
Precio 15.555	Indicar un máximo de dos decimales.
Cantidad negativa o 2.5	Rechazar la cantidad.
Cuaderno, 15.50 y 2	Registrar y mostrar total Bs 31.00.
Cantidad 0	Aceptar y mostrar total cero.
Pulsar Escape o Limpiar	Vaciar campos y conservar la lista.
Eliminar uno de dos nombres iguales	Retirar solo el objeto seleccionado.
Recargar la página	Vaciar la lista almacenada en memoria.

Desarrollo e integración con Neon y Render

El desarrollo amplió el formulario inicial con una base de datos remota y un servidor que concentra las operaciones de acceso a los productos. Las figuras siguientes presentan la secuencia de creación de recursos, preparación del repositorio y publicación de la aplicación.

Persistencia de productos en PostgreSQL

La tabla productos contiene un identificador generado automáticamente, nombre, precio, cantidad y fecha de creación. Las restricciones impiden nombres vacíos, precios no positivos y cantidades negativas. El servidor utiliza consultas parametrizadas para enviar los valores por separado de las instrucciones SQL.

```
CREATE TABLE IF NOT EXISTS productos (
 id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 nombre TEXT NOT NULL CHECK (length(trim(nombre)) > 0),
 precio NUMERIC(12,2) NOT NULL CHECK (precio > 0 AND precio <> 'NaN'::numeric),
 cantidad INTEGER NOT NULL CHECK (cantidad >= 0),
 creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Comunicación entre la interfaz y el servidor

ApiProductos.js utiliza fetch para comunicarse con /api/productos. GET carga la lista, POST registra un producto y DELETE /api/productos/:id elimina el registro seleccionado. ControladorProductos convierte los datos recibidos en objetos Producto y actualiza la lista después de cada operación confirmada por el servidor.

Antes de eliminar se abre un cuadro de diálogo con el nombre del producto y las opciones Sí, eliminar y Cancelar. Cancelar o pulsar Escape cierra el cuadro sin enviar la solicitud DELETE. La conexión se guarda en DATABASE_URL del servidor y no se incorpora a los archivos públicos.

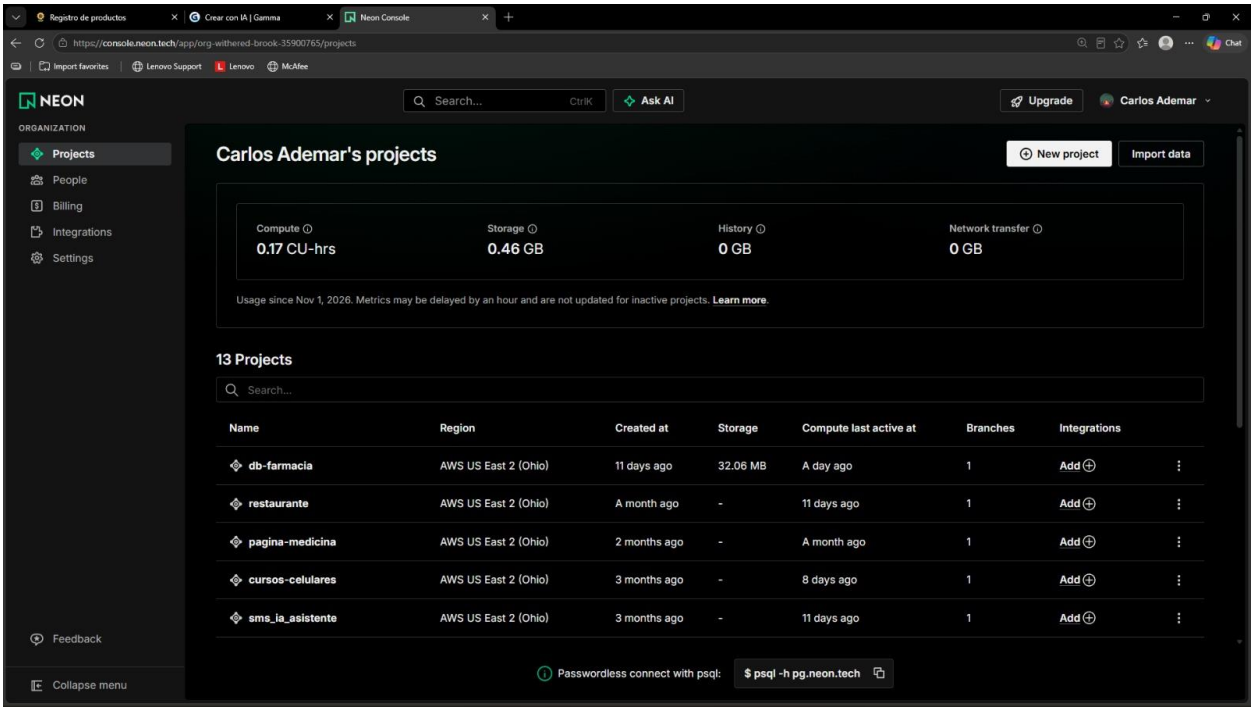
Preparación y publicación de la aplicación

El código se publicó en GitHub y se vinculó a un servicio web en Render. La construcción utiliza npm install. El error inicial de arranque fue node: .env: not found, porque el comando exigía un archivo local ausente en el despliegue. La configuración se corrigió para recibir DATABASE_URL desde el entorno y ejecutar el servidor sin exigir ese archivo. El servidor escucha en 0.0.0.0 y utiliza el puerto indicado por PORT.

Las capturas distinguen la instalación de dependencias, el despliegue y la vista final. La figura de Netlify documenta una alternativa explorada; la evidencia de publicación final corresponde a Render. El estado Live acredita el despliegue, pero no sustituye una prueba completa de todas las validaciones y operaciones.

Figura 1

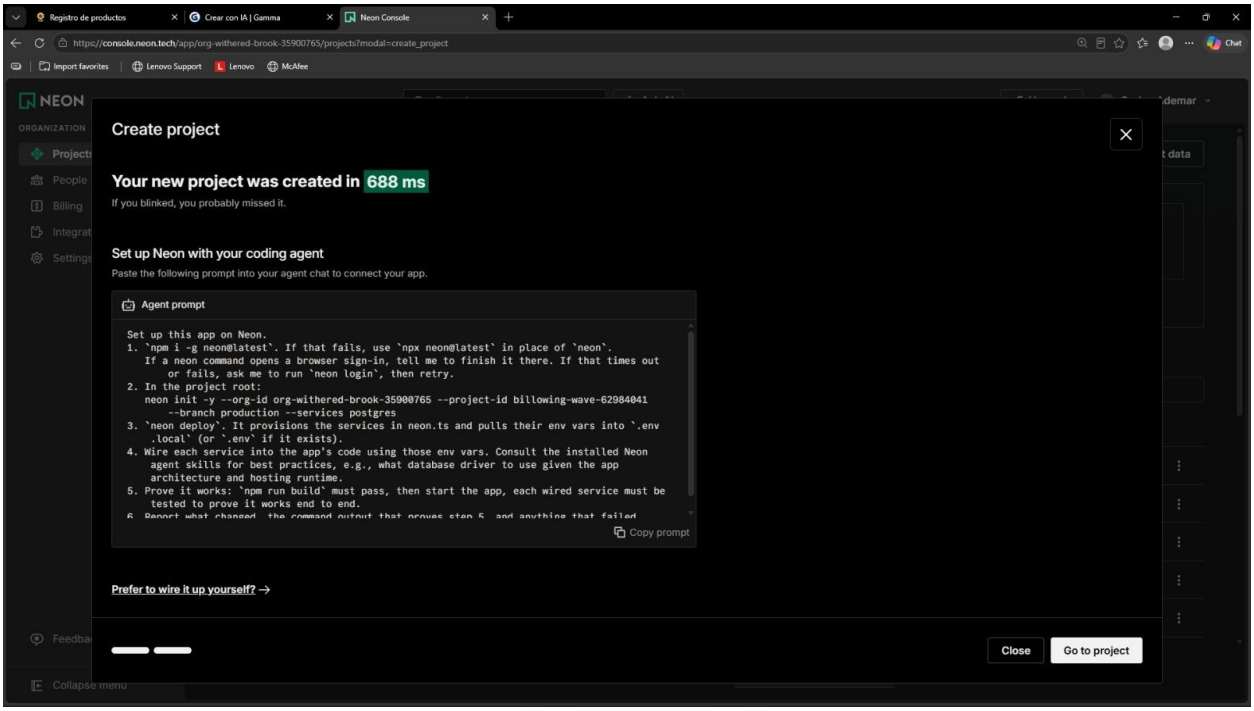
Panel de proyectos de Neon



Nota. Acceso al panel desde el que se organiza la base de datos del ejercicio. Captura proporcionada por el autor.

Figura 2

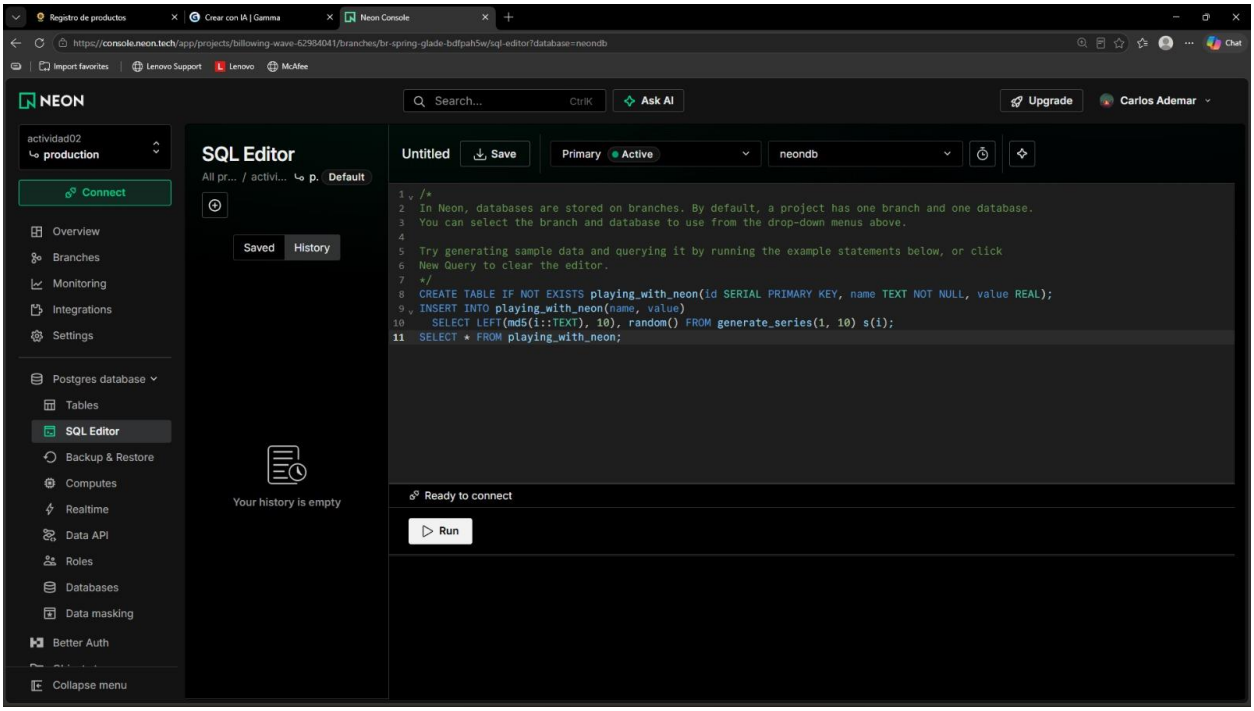
Creación del proyecto en Neon



Nota. Confirmación de creación del proyecto y opciones para iniciar su configuración. Captura proporcionada por el autor.

Figura 3

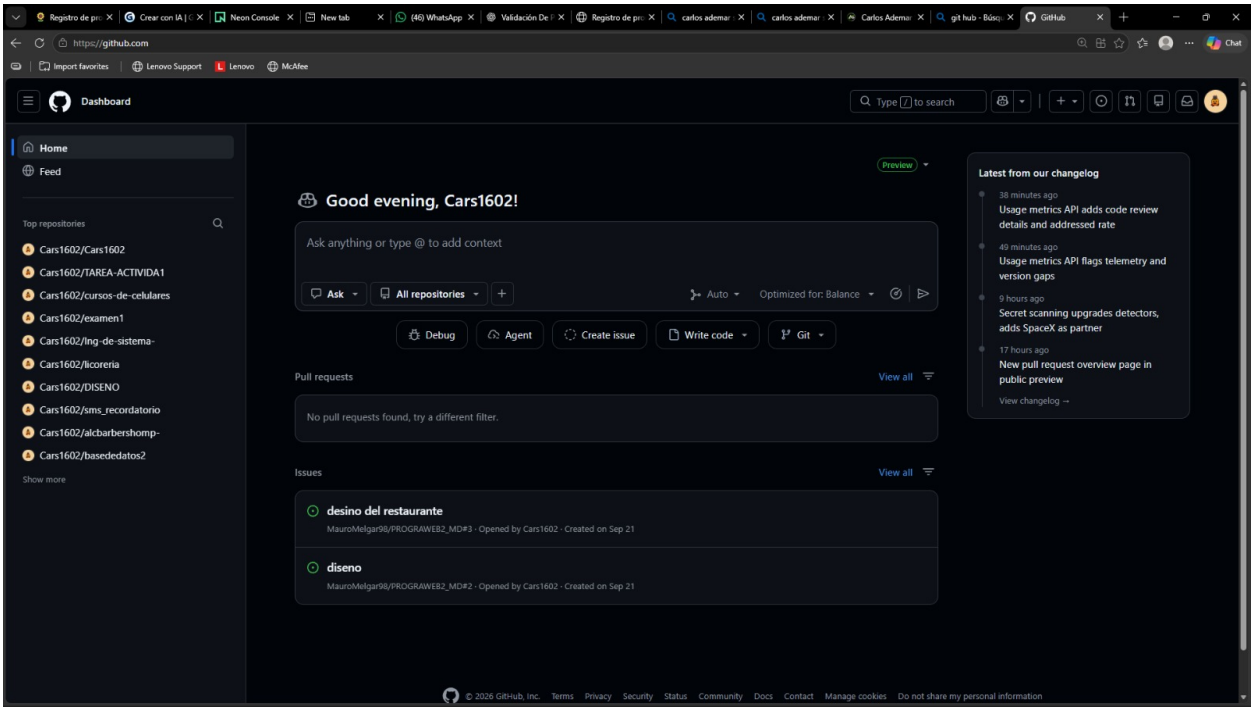
Editor SQL de Neon



Nota. Vista inicial del editor. El código visible es el ejemplo predeterminado de Neon, no el esquema final de productos. Captura proporcionada por el autor.

Figura 4

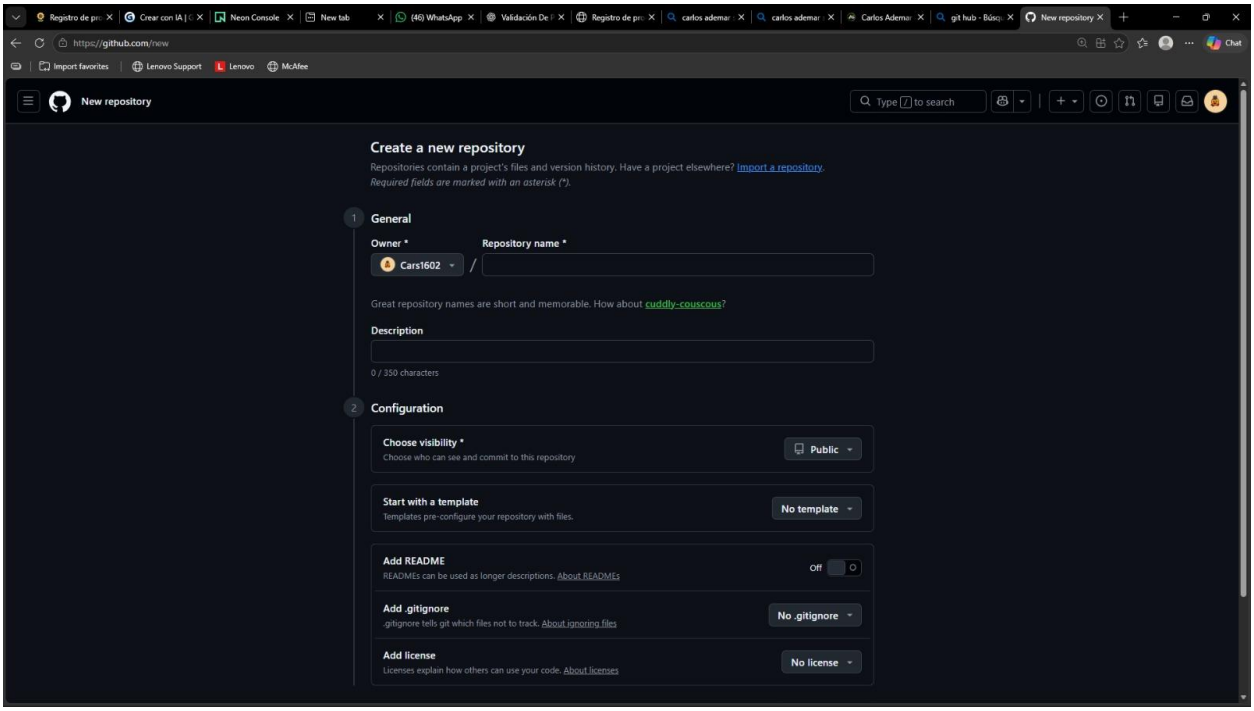
Acceso a GitHub



Nota. Panel utilizado para iniciar la publicación del código fuente. Captura proporcionada por el autor.

Figura 5

Creación del repositorio

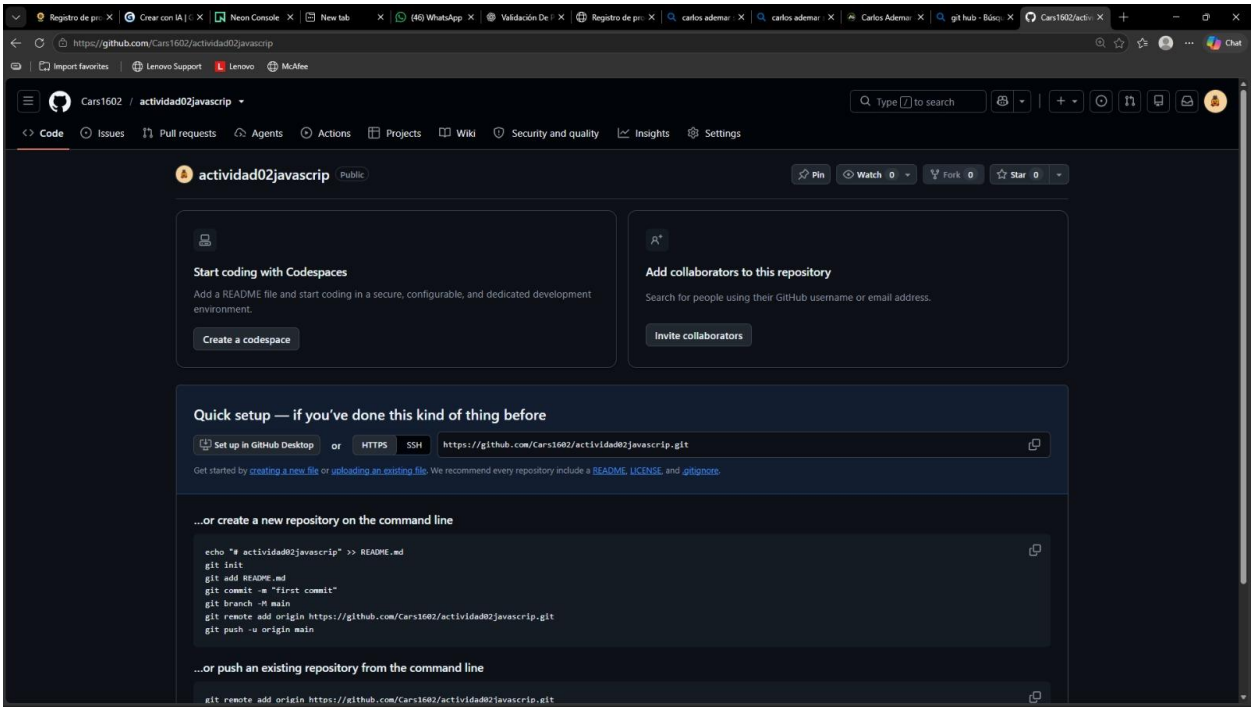


Nota. Formulario de creación de un repositorio para almacenar los archivos del proyecto.

Captura proporcionada por el autor.

Figura 6

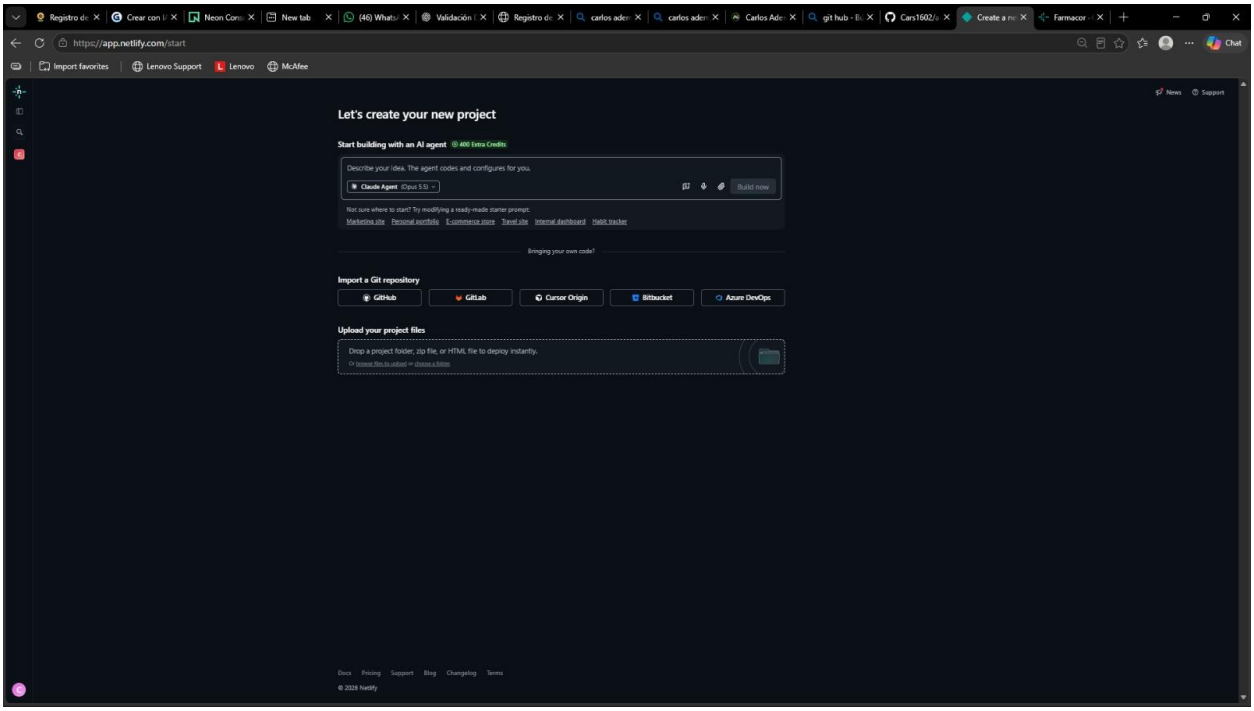
Repositorio del proyecto



Nota. Repositorio actividad02javascrip creado y preparado para recibir los archivos. Captura proporcionada por el autor.

Figura 7

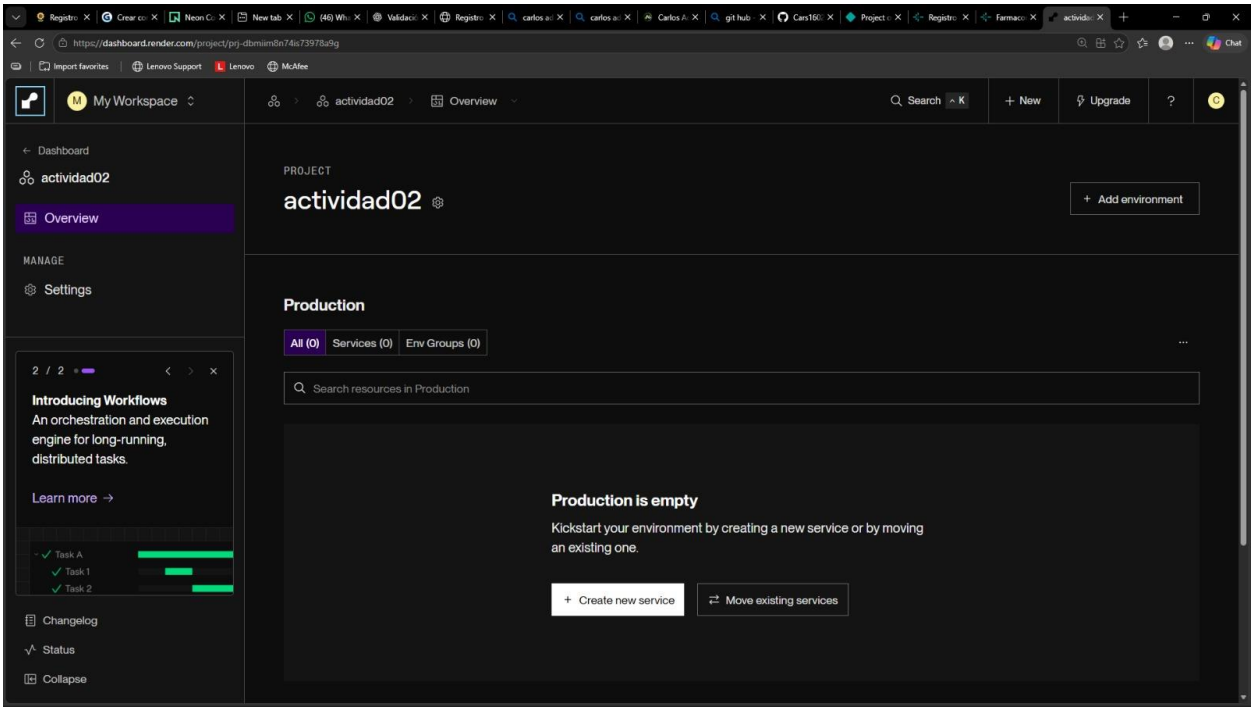
Alternativa de publicación en Netlify



Nota. Pantalla de importación explorada antes de la publicación final en Render. Captura proporcionada por el autor.

Figura 8

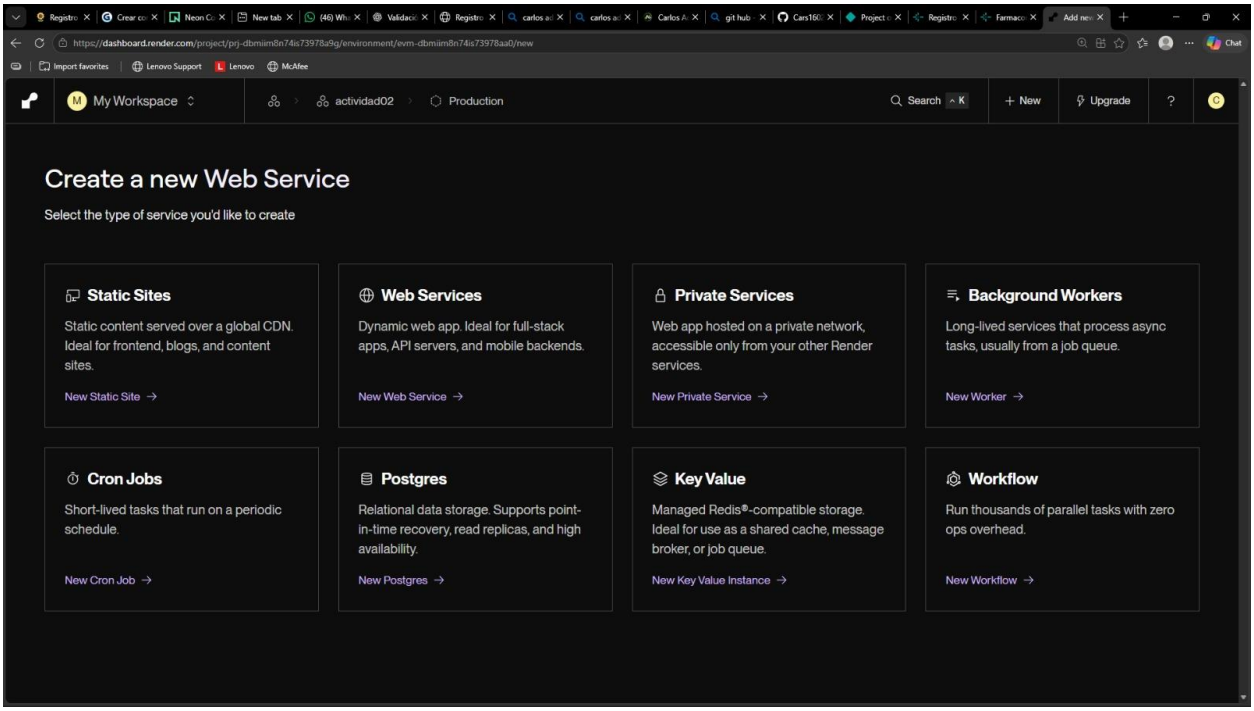
Proyecto en Render



Nota. Entorno inicial del proyecto actividad02 antes de agregar el servicio. Captura proporcionada por el autor.

Figura 9

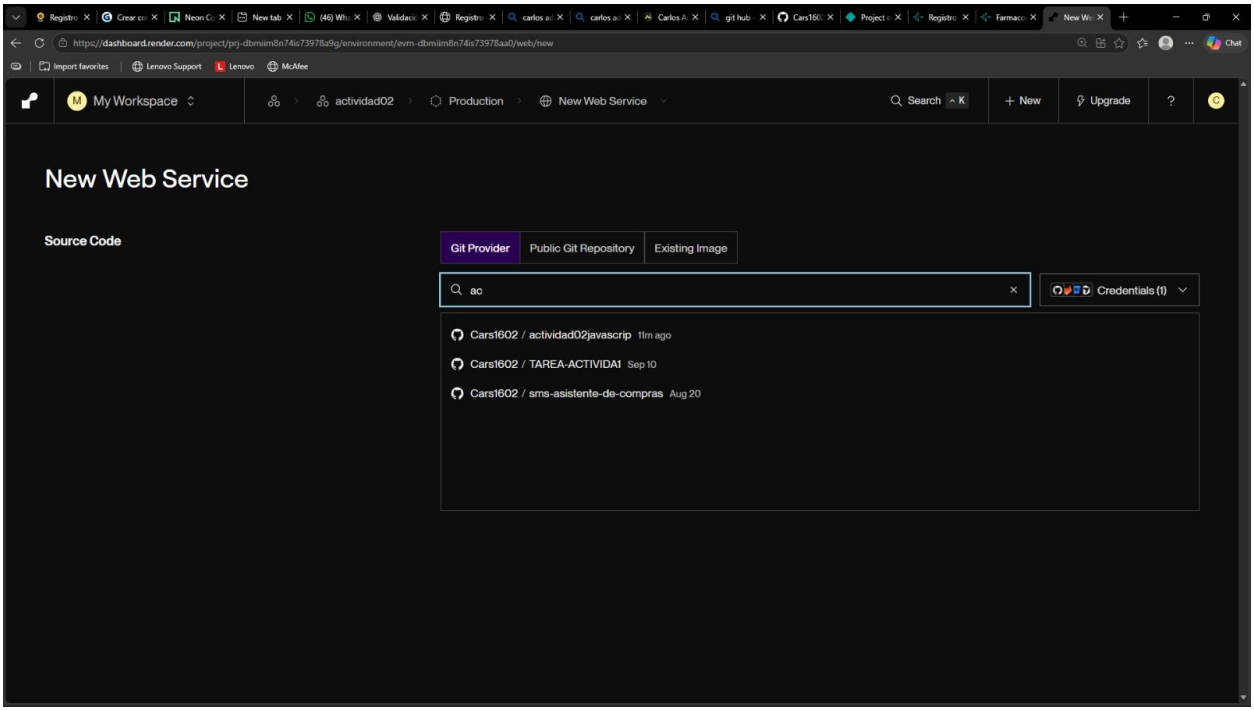
Selección del servicio web



Nota. Selección de Web Services para ejecutar el servidor Node.js. Captura proporcionada por el autor.

Figura 10

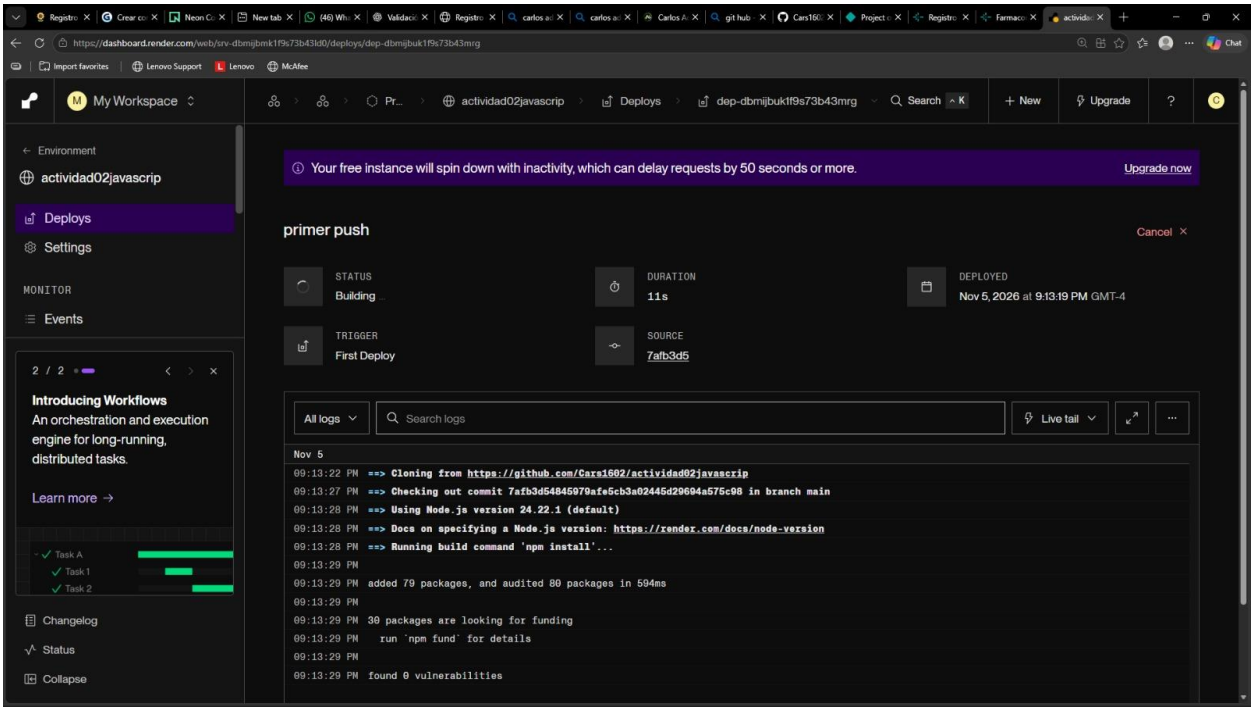
Vinculación de GitHub con Render



Nota. Selección del repositorio actividad02javascrip como origen del despliegue. Captura proporcionada por el autor.

Figura 11

Instalación de dependencias

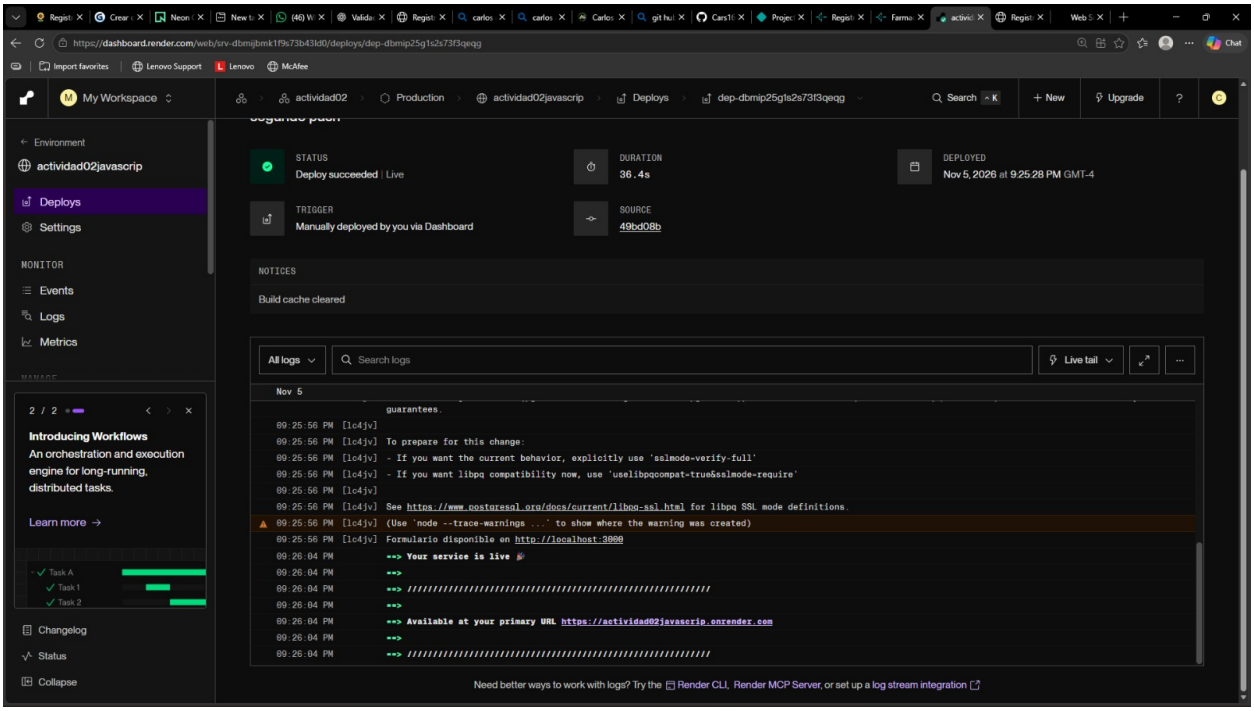


Nota. Los registros muestran npm install y la instalación de 79 paquetes en el primer intento.

Captura proporcionada por el autor.

Figura 12

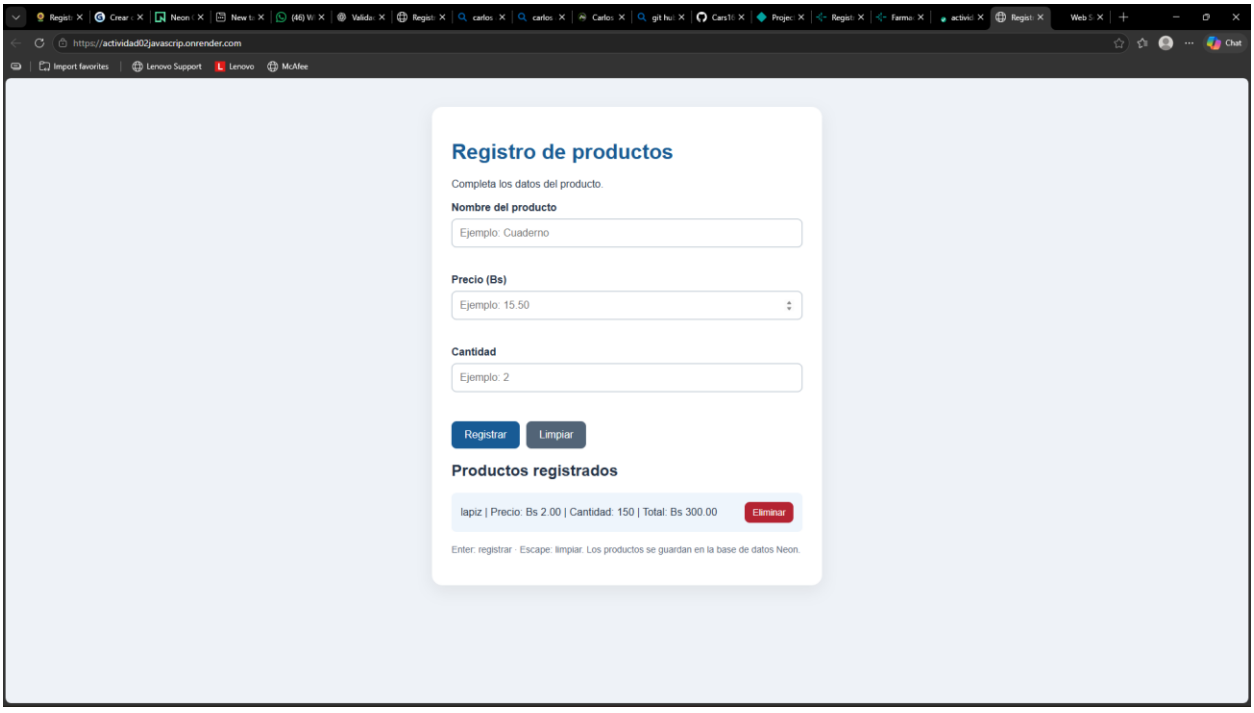
Despliegue exitoso en Render



Nota. El panel muestra Deploy succeeded y el mensaje Your service is live después de las correcciones. Captura proporcionada por el autor.

Figura 13

Aplicación publicada



Nota. La interfaz muestra un producto lápiz con precio Bs 2.00, cantidad 150 y total Bs 300.00. Captura proporcionada por el autor.

Limitaciones y mejoras

Una siguiente versión puede incorporar edición de productos, autenticación y autorización. La persistencia en Neon y la confirmación de eliminación ya forman parte de la implementación. Para operaciones monetarias de producción se debe definir una estrategia de redondeo y ampliar las comprobaciones del sistema.

Ejes transversales y conclusiones

Aprendizaje y uso responsable de inteligencia artificial

La asistencia de IA se utilizó en la conversación para proponer código comentado, dividirlo en módulos y añadir la eliminación individual. Comprender cada método sigue siendo una

parte central del taller: copiar código sin revisar los eventos y las reglas impediría explicar su funcionamiento o detectar errores.

Entre las solicitudes realizadas estuvieron iniciar el proyecto desde cero, separar el código en archivos con comentarios y permitir eliminar un producto registrado. Estas intervenciones orientaron la evolución del ejercicio; no sustituyen las comprobaciones manuales ni la lectura de documentación técnica.

Relación educativa y social

El proyecto se relaciona con el ODS 4 al apoyar el aprendizaje de competencias digitales y con el ODS 9 al practicar el desarrollo de soluciones tecnológicas. La claridad de los mensajes, las etiquetas asociadas a los campos y el uso del teclado favorecen una interfaz más accesible. Esta relación es una interpretación educativa del ejercicio, no una evaluación formal de impacto.

Conclusiones

La solución propuesta aplica programación orientada a objetos mediante clases que separan los datos del producto, las reglas de validación y el control de la interfaz. Esta división permite comprender y ampliar el programa de manera gradual.

Los eventos conectan las acciones del usuario con los métodos del controlador. La validación durante la escritura orienta la corrección y la revisión al enviar evita registrar datos incompletos. La eliminación individual mantiene sincronizados el arreglo y la lista visible.

El taller integra programación orientada a objetos, validación de formularios y persistencia en PostgreSQL. Las capturas documentan la creación de recursos, la publicación del código y un despliegue exitoso en Render, además de la interfaz publicada con un producto registrado.

Referencias

MDN Web Docs. (s. f.-a). Clases.

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes

MDN Web Docs. (s. f.-b). Element.addEventListener.

https://developer.mozilla.org/es/docs/Web/API/EventTarget/addEventListener

MDN Web Docs. (s. f.-c). Módulos JavaScript.

https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Modules